

Assignment One

CS 498

Total: 79 points

Instructions

Submit your responses to the short-answer questions as a PDF, together with `q2.py`, `q3.py`, and `q4.py`, on Canvas. Include your name and NetID in your written responses. Show your calculations and briefly explain your reasoning where requested.

For Coding Q2, Q3, and Q4, use the starter code, test runners, and instructions in the [CS498 homework repository](#). Follow the homework `README.md` for all three coding questions.

Coding grading

Each coding question is graded **30% by public tests and 70% by hidden tests**. This split also applies within each coding component. Short-answer grading is unchanged.

Coding question	Total	Public (30%)	Hidden (70%)
Q2	10	3	7
Q3	30	9	21
Q4	25	7.5	17.5

Run `python grade_public.py --question 2 --submission .` from the homework folder, using 3 or 4 for the other questions. There are three public and seven hidden cases per question; each contributes 10% of each component's points. Hidden cases stay within the documented requirements.

In addition to the public and hidden tests, course staff will run your submitted implementations with our official copy of `run.py`. Q2/Q3 must complete Qwen training and pass the runner's checks; Q4 must pass its component and combined forward/backward checks. If your implementation fails the official `run.py` checks, **50% of the affected coding question's total points will be deducted: 5 points for Q2, 15 points for Q3, or 12.5 points for Q4**. This deduction is applied once per affected question, after combining its public and hidden test scores, with a minimum final score of zero. Multiple failed checks within the same question do not incur additional runner-failure deductions. These checks do not award additional points beyond the stated allocation.

See `README.md` for scoring reports and execution instructions. Official grading uses staff-controlled runners and test files with your submitted `q2.py`, `q3.py`, and `q4.py`. Do not submit or modify grading files. Code inspection enforces implementation restrictions; it does not add points outside the 30/70 split.

1. Short Answer Questions — 14 points

Q1. (1 point)

A transformer has a model hidden dimension of 768 and 12 equal-sized attention heads. What is the hidden dimension of each head?

Q2. (1 point)

During autoregressive decoding with a KV cache, a newly generated token attends to a context of s tokens. What is the time complexity of its self-attention computation in terms of s , holding the number of layers, number of heads, and head dimension fixed?

Q3. (2 points)

Consider a standard multi-head self-attention layer with 16 attention heads and a sequence length of 1,024 tokens, processing one sequence. Assume each head explicitly stores its full attention-score matrix, including masked positions.

- (a) How many attention-score entries are stored across all heads? **(1 point)**
- (b) If the sequence length doubles, by what factor does the memory required for these matrices increase, assuming the datatype remains unchanged? **(1 point)**

Q4. (2 points)

A transformer has $L = 61$ layers, $h = 128$ attention heads per layer, and head dimension $d_k = 128$. What is the total number of scalar key and value elements cached **per token across all layers** when using (a) multi-head attention (MHA) and (b) multi-query attention (MQA)? Assume key and value head dimensions are equal, and MQA shares one key head and one value head across all query heads. **(1 point each)**

Q5. (4 points)

Consider a standard MHA transformer with $L = 64$ layers, model hidden dimension $d_{\text{model}} = 8192$, $h = 32$ heads per layer, head dimension $d_k = 256$, and **vocabulary size $V = 65,536$** . Each layer has a two-matrix MLP with an up-projection factor of 4 and a down-projection back to d_{model} .

Calculate the total number of parameters and the memory required to store these weights in bf16 (2 bytes per parameter). Count the input token embedding matrix, the query/key/value and attention output projections, and the MLP matrices. Exclude the final vocabulary output projection, biases, layer normalization, and positional encoding parameters. Report memory in GiB, where $1 \text{ GiB} = 2^{30}$ bytes.

Q6. (4 points)

Consider the following transformer MLP:

$$Y = \text{GeLU}(XW_1)W_2$$

The input X has shape 128×4096 , W_1 has shape 4096×16384 , and W_2 has shape 16384×4096 . Consider only the forward pass and ignore biases and dropout.

The MLP uses tensor parallelism across four GPUs. Every GPU starts with the same X . Partition W_1 evenly by columns and W_2 evenly by corresponding rows.

- (a) What are the shapes of the local W_1 and W_2 shards on each GPU? **(1 point)**

- (b) Consider an alternative partition for the first matrix multiplication only: W_1 is partitioned by rows, and each GPU computes a partial product using the corresponding columns of X . Must these partial products be summed before or after applying GeLU to reproduce the original MLP computation? Explain briefly. **(1 point)**
- (c) Returning to the original column-wise partition of W_1 , what is the shape of each GPU's local intermediate activation after GeLU? Does computing this activation require communication between GPUs? Explain briefly. **(1 point)**
- (d) After multiplying the local activation by the corresponding W_2 shard, which collective operation produces the complete output Y on every GPU? Specify the reduction operation and explain why it is needed. **(1 point)**

2. Parameter Server — 10 points

Implement synchronous data-parallel training with a centralized parameter server in `q2.py`. Three CPU processes hold full copies of Qwen3-0.6B and compute gradients on different samples. Rank 0 also acts as the server: it contributes its own gradients, updates the model, and distributes the new parameters.

The provided `run.py` handles data loading and gradient computation. It then calls `PS_grads_`, which selects parameters with non-None gradients and dispatches to your server or worker function. Implement the missing communication and aggregation logic; preserve the existing function signatures.

(a) Server — 5 points. Complete `server(params, opt, world)` on rank 0. Receive gradients from every other rank, include rank 0's own gradients, and compute their elementwise average over all ranks. Use this average for the provided optimizer step, then send the updated parameters to every worker. The optimizer must advance exactly once per training step on rank 0.

(b) Worker — 5 points. Complete `worker(params)` on each nonzero rank. Send the local gradients to rank 0, receive the updated parameters, and copy them into the local model with their original shapes and ordering. Workers must use the server's updated parameters rather than apply their own optimizer step.

Implementation requirements

Implement gradient and parameter transfers with non-blocking point-to-point operations, **`dist.isend`** and **`dist.irecv`**. Wait for communication to complete before reading received data or modifying/reusing a send buffer. Do not use built-in collectives such as `all_reduce`, `reduce`, `gather`, or `broadcast` to implement these transfers. The barrier already supplied in the wrapper may remain.

Use the provided `flatten/unflatten` helpers to pack tensors and restore their shapes, with the same parameter ordering on every rank. After synchronization, all ranks must hold the same updated parameters. Only rank 0 applies optimizer updates and maintains the active optimizer state.

Execution and testing

Follow the homework README to install dependencies and configure the three-node Gloo environment. Use the same `netid.txt` and `MASTER_ADDR` on all nodes and select each node's experiment-network interface with `GL00_SOCKET_IFNAME`. Model and dataset loading requires internet access or cached files.

Run the following command on **each of the three nodes**, replacing `RANK` with 0, 1, or 2 respectively:

```
bash glooHelper.sh -n 3 -P 1 -r RANK -e 1 -q 2 -s ./run.py
```

Use `-e 0` first to check initialization and loading; this does not test synchronization. Training checks that rank 0's parameters change and the final models match across ranks. Matching models alone does not establish correct gradient averaging; verify the specified update behavior too.

Submit: `q2.py`. You may modify `run.py` for your own testing, but do not submit it. Coding Q2 is independent of Coding Q3 and Q4.

3. Ring All-Reduce — 30 points

Implement decentralized gradient synchronization in `q3.py`. As in Coding Q2, three CPU processes train full copies of Qwen3-0.6B on different data. Here, the ranks exchange gradient chunks around a logical ring so that every rank obtains the same averaged gradients and can apply its own optimizer update.

The provided `run.py` flattens the model gradients into one tensor, calls **`ring_allreduce_`**, restores the synchronized gradients, and applies the optimizer step on every rank. Your code implements the tensor synchronization, not the optimizer update.

(a) All-reduce — 10 points. Complete **`ring_allreduce_(tensor, world_size, rankid)`**. Divide the flattened tensor into equally sized chunks, adding zero padding when its length is not divisible by the number of ranks. Call your reduce-scatter and all-gather phases, average the resulting sums across ranks, remove padding, and write the result back into the original tensor **in place**. Preserve its shape and length. A single-rank call should leave the tensor unchanged.

(b) Reduce-scatter — 10 points. Complete **`reduce_scatter`**. Exchange chunks with neighboring ranks and accumulate contributions so that, after this phase, each rank owns one different chunk containing the elementwise sum of that chunk across all ranks. Keep chunk ownership consistent with your all-gather implementation.

(c) All-gather — 10 points. Complete **`all_gather`**. Circulate the completed chunks around the ring so every rank obtains all reduced chunks in their original order. This phase distributes the reduced values; it must not sum them again.

Implementation requirements

Use non-blocking point-to-point **`dist.isend`** and **`dist.irecv`** between ring neighbors, waiting for operations to finish before reading or reusing their buffers. Do not replace the algorithm with built-in `all_reduce`, `reduce_scatter`, `all_gather`, or other data-transfer collectives. Collectives already used by the runner for setup and correctness checks are outside this restriction.

Keep the `ring_allreduce_` interface unchanged. You may adjust the helper signatures of `reduce_scatter` and `all_gather`, provided you update their calls consistently. Ensure that the final copy-back uses the reduced, averaged data, including when padding creates a separate buffer. Average exactly once: unlike Coding Q4's partial outputs, the gradients in this question must be averaged.

Execution and testing

Use the same three-node setup as Coding Q2. Run the following on every node, replacing `RANK` with 0, 1, or 2 respectively:

```
bash glooHelper.sh -n 3 -P 1 -r RANK -e 1 -q 3 -s ./run.py
```

The final check in `run.py` verifies that all model copies match. Also check small tensors with known averages, including a length not divisible by three, to verify arithmetic and padding. For example, rank inputs `[1, 2, 3, 4]`, `[4, 5, 6, 7]`, and `[7, 8, 9, 10]` should become `[4, 5, 6, 7]` on every rank.

Submit: `q3.py`. You may modify `run.py` for local testing, but do not submit it. Your implementation must not depend on your Coding Q2 solution.

4. Tensor-Parallel MLP — 25 points

Implement both forward and backward computation for a tensor-parallel MLP. Q3 synchronized gradients of full model replicas; Q4 partitions a layer's weights and combines partial outputs and input gradients. This question is independent of your Q2/Q3 implementations.

Complete the four functions in `q4.py`. Use CPU float32 tensors and PyTorch Gloo. Ignore biases and dropout. No optimizer update is required. The full MLP is:

$$Y = \text{GeLU}(XW_1)W_2$$

X has shape $[T, H]$, W_1 has shape $[H, F]$, and W_2 has shape $[F, H]$. All dimensions are positive. There are P ranks, numbered 0 through $P-1$, and $F \geq P$. F need not be divisible by P . Every rank receives the same X . The runner provides full weights for setup and reference checks; your forward and backward functions receive only local shards.

(a) Weight partitioning — 5 points

Implement **`shard_weights(w1, w2, rank, world_size)`**. Partition W_1 by columns and W_2 by matching rows. Let $q = \text{floor}(F/P)$ and $m = F \bmod P$. The first m ranks own $q+1$ features each, and the remaining ranks own q features each, assigned contiguously in rank order. Rank r starts at feature $r \cdot q + \min(r, m)$.

Return independent, contiguous tensors of shapes $[H, F_r]$ and $[F_r, H]$, where F_r is the local width. Do not modify the full weights or retain their storage through views. For $F=10$ and $P=3$, the widths are 4,3,3 and the starting offsets are 0,4,7. Empty shards are not required.

(b) Distributed forward pass — 10 points

Implement **`sum_across_ranks(tensor, rank, world_size)`**. Rank 0 receives and sums all contributions, including its own, then sends the result to every other rank. Return a fresh result on every rank without modifying the input. Sum; do not average. For one rank, return an independent copy without communication.

Implement **`mlp_forward(x, w1_local, w2_local, rank, world_size)`**. Compute $Z_r = XW_{1,r}$, $A_r = \text{GeLU}(Z_r)$, and $Y_r = A_r W_{2,r}$. Call your sum helper exactly once to produce the full Y . Use the default exact `torch.nn.functional.gelu`. Return (Y, cache) , with `cache = (x, z, a, w1_local, w2_local)`. Do not modify inputs; cached references are allowed.

Communication requirements

Use only **`dist.isend`**, **`dist.irecv`**, and request **`wait()`** for rank-0 aggregation and distribution. Built-in collectives, barriers, blocking send/recv, and reuse of Q2/Q3 code are not allowed in your solution. Only `sum_across_ranks` may communicate. The runner initializes the process group and supplies rank and world size.

Keep buffers alive until their requests complete. Wait before reading received data, modifying send buffers, or returning. Repeated calls during forward and backward must not mix messages. Separate tags for aggregation and distribution are allowed.

(c) Manual backward pass — 10 points

Implement `mlp_backward(grad_output, cache, rank, world_size)`, returning `(grad_x, grad_w1_local, grad_w2_local)`. Let $G = \text{grad_output}$, the same $[T, H]$ gradient on every rank. Use these supplied formulas; deriving them is not required.

Below, $X, Z, A, W1$, and $W2$ refer to the tensors in your local cache (`x, z, a, w1_local, w2_local`). Let F_{local} be this rank's shard width. $@$ means matrix multiplication, $.T$ means transpose, and $*$ means elementwise multiplication.

Computation	Result shape
$dW2 = A.T @ G$	$[F_{\text{local}}, H]$
$dA = G @ W2.T$	$[T, F_{\text{local}}]$
$dZ = dA * \text{gelu_derivative}(Z)$	$[T, F_{\text{local}}]$
$dW1 = X.T @ dZ$	$[H, F_{\text{local}}]$
$dX_{\text{local}} = dZ @ W1.T$	$[T, H]$
$dX = \text{sum_across_ranks}(dX_{\text{local}}, \text{rank}, \text{world_size})$	$[T, H]$

Return `(dX, dW1, dW2)`. The starter provides `gelu_derivative`; do not modify it. Call your sum helper exactly once to obtain `dX`. **Weight gradients stay local:** do not reduce `dW1` or `dW2`, or divide `G` by the number of ranks. Preserve the incoming gradient and cache tensors, and use the pre-update weights. Do not use `autograd`, `.backward()`, `torch.autograd.grad`, or custom `autograd` functions; implement the equations directly.

Execution, testing, and submission

Use the same three-machine CloudLab/Gloo setup as Q2 and Q3. Install CPU PyTorch and copy the same homework files and completed `q4.py` to each machine. Use identical `netid.txt` contents and `MASTER_ADDR`, and set each machine's `GL00_SOCKET_IFNAME`. No GPU, pretrained model, or dataset is required.

Run the following on **each machine**, replacing `RANK` with 0, 1, or 2:

```
bash glooHelper.sh -n 3 -P 1 -r RANK -q 4 -s ./run.py
```

Q4 has no epochs, so `-e` is not needed. Pass test options after `--`, using the same options on all machines; for example, append `--component backward`. The helper uses a shared NetID-derived port; any `MASTER_PORT` override must be identical across machines. For optional local debugging, use `torchrun --standalone --nproc-per-node=3 run.py --question 4`. Public scoring still runs locally through `grade_public.py`.

Also test locally with 1 and 4 processes. Use `--component shard, sync, forward, or backward` to test individual parts. Forward/backward component tests supply a reference sum helper, and backward tests supply a reference cache. The combined forward/backward checks use all of your own functions. The runner's reference `autograd` and collectives are allowed; those shortcuts are forbidden in `q4.py`.

Public tests cover uneven partitions, one-feature shards, repeated communication, and combined forward/backward correctness. Numerical comparisons use `rtol=1e-4` and `atol=1e-5`. Grading combines correctness checks with code inspection. Follow the function contracts above and see

README.md for setup and scoring details.

Submit: q4.py. Preserve function signatures, the provided derivative helper, and no-grad decorators. Do not modify the runner, reconstruct full weights during forward/backward, or hard-code test inputs. Use run.py --question 4 for Q4 diagnostics; --question 2 and --question 3 select Q2/Q3 training.